Chapter 9

9. Regression models.

- 9.1. Tuning the model.
- 9.2. Shrinkage methods.
- 9.3. Extensions.
- 9.4. Principal components regression.
- 9.5. Issues.

Tuning the model

We consider a regression problem where $Y \sim N_n(\mu, \sigma^2 I_n)$. We assume that μ depends linearly on regressor variables $x_i \in \mathbb{R}^n$, $1 \leq i \leq p$.

The x_i are called the covariates, predictors or explanatory variables and Y is the response or dependent vector.

A few important questions we want to answer are:

1. Is there at least one important predictor among the x_i's?
2. If the answer is yes in Question 1, which among the x_i's are important?
3. How well does the model fit?

Tuning the model

Question 1:

Is there at least one important predictor among the x_i's?

To answer this question we can use the F statistic. We test a model with constant mean $L_2 = \mathrm{sp}(1_n)$ against the full model $L_1 = \mathrm{sp}(1_n, x_1, \ldots, x_p)$.

The F statistic is

$$F(Y) = \frac{\|\hat{\mu}_1 - \hat{\mu}_2\|^2/p}{\mathrm{MSR}},$$

where

$$\mathrm{MSR} = \frac{1}{n-p-1}\|Y - \hat{\mu}_1\|^2.$$

Under H_0 we have $F(Y) \sim F_{p,n-p-1}$.

Tuning the model

In the example from the last chapter:

```
> lin.mod=lm(volume~1+diameter+hight)
> summary(lin.mod)

.....

Residual standard error: 3.896 on 28 degrees of freedom
Multiple R-squared: 0.9475,     Adjusted R-squared: 0.9438
F-statistic: 252.9 on 2 and 28 DF,  p-value: < 2.2e-16
```

Tuning the model

Question 2:

If yes in Question 1, which among the x_i's are important?

Naturally we will look at the individual p-values associated to the t-tests for each covariate.

```
Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept)  -1.64077    0.70021  -2.343   0.0264 *
log.diameter  1.97824    0.07614  25.983  < 2e-16 ***
log.height    1.13313    0.20751   5.461 7.89e-06 ***
---
Signif. codes:  0 *** 0.001 ** 0.01 * 0.05 . 0.1   1
```

This amounts to a multiple testing problem, and if p is large, it is likely that we will make false discoveries.

Tuning the model

Note also, that adding or removing variables can impact other variables. E.g. adding a variable, may make a non-significant variable significant and vice versa. Here is an example with the PM10 data.

```
# We fit two linear models
# Model 2 has additional variable

mod1 <- lm(PM.GR~PM.GR.LAG+T.KL, data=pm.daily)
mod2 <- lm(PM.GR~PM.GR.LAG+T.KL+T.GR, data=pm.daily)
```

Tuning the model

```
> summary(mod1)

Call:
lm(formula = PM.GR ~ PM.GR.LAG + T.KL)

Residuals:
    Min      1Q  Median      3Q     Max
-49.441  -7.678  -1.528   6.173  84.748

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) 13.69015    2.54970   5.369 2.43e-07 ***
PM.GR.LAG    0.70559    0.05089  13.866  < 2e-16 ***
T.KL        -0.37602    0.19218  -1.957    0.052 .
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 15.38 on 179 degrees of freedom
Multiple R-squared:  0.5345,Adjusted R-squared:  0.5293
F-statistic: 102.8 on 2 and 179 DF,  p-value: < 2.2e-16
```

T.KL is not significant at 5% level.

Tuning the model

```
> summary(mod2)

Call:
lm(formula = PM.GR ~ PM.GR.LAG + T.KL + T.GR, data = pm.daily)

Residuals:
    Min      1Q  Median      3Q     Max
-45.494  -7.291  -0.236   7.056  76.812

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) 21.03813    3.05912   6.877 9.97e-11 ***
PM.GR.LAG    0.61755    0.05358  11.525  < 2e-16 ***
T.KL         1.49464    0.50195   2.978  0.00331 **
T.GR        -2.31364    0.57732  -4.008 9.01e-05 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 14.77 on 178 degrees of freedom
Multiple R-squared:  0.573,Adjusted R-squared:  0.5658
F-statistic: 79.62 on 3 and 178 DF,  p-value: < 2.2e-16
```

T.KL is significant at 1% level.

Tuning the model

So ideally, we should try all 2^p submodels and check which one is best. Of course, with large p this is practically not feasible!

Hence most commonly, one of the following techniques is used:

Forward selection:

1. Start with the model which contains only an intercept.
2. Fit p regression models on the spaces $\mathrm{sp}(1, x_i)$.
3. Choose the one which leads to smallest SSR (equivalently highest R^2=1-SSR/SST).
4. Add one of the $p-1$ remaining explanatory variables, and check again for the best fit.
5. Repeat until a stopping rule is satisfied.

Tuning the model

Backward selection:

1. Start with all variables in the model.
2. Remove the variable with the largest p-value.
3. Fit the model with $p - 1$ variables, and remove the one with largest p-value. Exercise: this is equivalent to say that we remove the one, which after removing gave the smallest increase in SSR.
4. Repeat until a stopping rule is satisfied. E.g. when all p-values are below a threshold.

Note: Backward selection cannot be used if $p > n$, while forward selection can always be used.

Tuning the model

Mixed selection:

1. Start with the model which contains only an intercept.
2. Proceed as in forward selection until a threshold is exceeded.
3. Now remove the variable with largest p-value.
4. We continue until all variables have sufficiently small p-value, and all variables outside the model would have a too large p-value if added to the model.

Tuning the model

Question 3:

How well does the model fit?

If we measure model fit in terms of R^2 we get an in-sample error, which decreases when adding more variables. For example, in a forward selection the best fit will be at the last step, when most variables are included.

This doesn't necessarily imply that the model is good! For the model fit we should use measures which take into account the number of variables.

Tuning the model

Common measures are:

Mallows' C_p: Suppose we have a model M with $k \leq p$ variables, then we set

$$C_p(M) = \frac{1}{n}(\mathrm{SSR}(M) + 2k\tilde{\sigma}^2),$$

where $\tilde{\sigma}^2 = \mathrm{MSR}(F)$ is the unbiased estimator of the variance in the full model F and $\mathrm{SSR}(M)$ is the sum of squared residuals in the model M.

The preferred model is the one with smallest $C_p(M)$.

C_p estimates the test MSE, and it seems natural that this shall be minimized.

Tuning the model

AIC (Akaike information criterion): Suppose we have a model M with k variables, then we set

$$\mathrm{AIC(M)} = 2k - 2\log L(\hat{M}).$$

Here $L(\hat{M})$ is the maximum of the likelihood using model M. In our Gaussian regression case this amounts (up to an additive constant) to

$$\widetilde{\mathrm{AIC}}(\mathrm{M}) = n\log(\mathrm{SSR(M)}/n) + 2k$$

The preferred model is the one with smallest AIC(M).

Can be shown to be equivalent to C_p (choosing same model).

Tuning the model

BIC (Bayesian information criterion): Suppose we have a model M with k variables, then we set

$$n \log(\text{SSR(M)}/n) + k \log(n).$$

The preferred model is the one with smallest $AIC(M)$.

Includes larger penalty for number of variables, hence prefers smaller models.

Tuning the model

Adjusted R^2: Suppose we have a model M with k variables, then we set

$$\text{Adj } R^2 = 1 - \frac{\text{SSR}/(n-k-1)}{\text{SST}/(n-1)}.$$

The preferred model is the one with largest Adj R^2.

While AIC, BIC and C_p have a theoretical (large sample) background, this is not the case for Adj R^2.

Tuning the model

We can now choose the "best model" in our different variable selection algorithms.

Forward/Backward selection: for each of the models M_k with k variables, compute the chosen criterion and then choose the model $M_{\hat{k}}$ with minimal AIC, BIC or C_p or maximal Adj R^2.

Alternatively, we choose our model via cross-validation or the validation set approach, which are nowadays with enhanced computational resources, common approaches.

Tuning the model

```
# load ISLR package for Auto data
library(ISLR)
# load leaps package for subsetregression
library(leaps)
# load xtable package to produce a nice table
library(xtable)
# apply subset regression, we exclude the variables "origin" and "name"
mod=regsubsets(mpg~.,data=Auto[,-c(8,9)],method="forward")
sum.mod=summary(mod)
# show the attributes we can extract
names(sum.mod)
[1] "which"  "rsq"    "rss"    "adjr2"  "cp"     "bic"    "outmat" "obj"
# show how the variable were included
sum.mod$which
```

	(Intercept)	cylinders	displacement	horsepower	weight	acceleration	year
1	TRUE	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE
2	TRUE	FALSE	FALSE	FALSE	TRUE	FALSE	TRUE
3	TRUE	FALSE	FALSE	FALSE	TRUE	TRUE	TRUE
4	TRUE	FALSE	TRUE	FALSE	TRUE	TRUE	TRUE
5	TRUE	TRUE	TRUE	FALSE	TRUE	TRUE	TRUE
6	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE

The variables were included in the order "weight", "year", "acceleration", "displacement", "cylinders" and "horsepower".

Tuning the model

The variables were included in the order `weight`, `year`, `acceleration`, `displacement`, `cylinders` and `horsepower`. We look at Mallows' C_p values for each of the models.

```
> sum.mod$cp
[1] 232.396144   1.169751   2.284200   3.992019   5.000800   7.000000
```

The smallest value is obtained with 2 variables (`weight`, `year`).

```
lm(formula = mpg ~ weight + year, data = Auto)

Residuals:
    Min      1Q  Median      3Q     Max
-8.8505 -2.3014 -0.1167  2.0367 14.3555

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept) -1.435e+01  4.007e+00  -3.581 0.000386 ***
weight      -6.632e-03  2.146e-04 -30.911  < 2e-16 ***
year         7.573e-01  4.947e-02  15.308  < 2e-16 ***
---

Residual standard error: 3.427 on 389 degrees of freedom
Multiple R-squared:  0.8082,Adjusted R-squared:  0.8072
F-statistic: 819.5 on 2 and 389 DF,  p-value: < 2.2e-16
```

Tuning the model

Slightly more handy is the function `stepAIC` from the package `MASS`.

```
> stepAIC(mod,
+         direction="forward",
+         scope=list(lower=mod, upper=Auto[,-c(8,9)]))
Start:  AIC=1611.93
mpg ~ 1

               Df Sum of Sq     RSS    AIC
+ weight        1   16497.8  7321.2 1151.5
+ displacement  1   15440.2  8378.8 1204.4
+ horsepower    1   14433.1  9385.9 1248.9
+ cylinders     1   14403.1  9415.9 1250.1
+ year          1    8027.7 15791.3 1452.8
+ acceleration  1    4268.5 19550.5 1536.5
<none>                      23819.0 1611.9
```

Tuning the model

```
Step:  AIC=1151.49
mpg ~ weight

               Df Sum of Sq    RSS     AIC
+ year          1   2752.28 4569.0  968.66
+ horsepower    1    327.39 6993.8 1135.56
+ acceleration  1    168.34 7152.9 1144.37
+ displacement  1    150.93 7170.3 1145.33
+ cylinders     1    115.12 7206.1 1147.28
<none>                      7321.2 1151.49

Step:  AIC=968.66
mpg ~ weight + year

               Df Sum of Sq    RSS    AIC
<none>                      4569.0 968.66
+ acceleration  1   10.4503 4558.5 969.77
+ cylinders     1    4.9581 4564.0 970.24
+ horsepower    1    3.3015 4565.7 970.38
+ displacement  1    0.0420 4568.9 970.66

Call:
lm(formula = mpg ~ weight + year, data = Auto)

Coefficients:
(Intercept)       weight         year
 -14.347253    -0.006632     0.757318
```

Tuning the model

```
# We load Graz pollution data
load("/Users/siegfriedhoermann/Dropbox/Data/PM10/Graz-Schadstoffe.Rdata")
# The dataframe graz contains pollution levels at different stations and
# meteorological variables and the date variable
names(graz)
[1] "datum"          "NO2_DB"         "NO2_Ost"        "NO2_Mitte"
[5] "PM10_DB"        "PM10_Ost"       "PM10_Mitte"     "lute"
[9] "lufe"           "wige"           "spg"            "nied"
[13] "nied01"         "ltus_gk"        "invers"         "frost"
[17] "wind"           "lute_lagged"    "lufe_lagged"    "wige_lagged"
[21] "spg_lagged"     "nied_lagged"    "nied01_lagged"  "ltus_gk_lagged"
[25] "invers_lagged"  "frost_lagged"   "wind_lagged"    "lute_diff"
[29] "lufe_diff"      "wige_diff"      "spg_diff"       "nied_diff"
[33] "ltus_gk_diff"
```

Tuning the model

```
# The function stepAIC needs the model fitting always on the same dataset.
# This may be a problem if there are missing values.
# It is suggest to remove the missing values.
graz=na.omit(graz)
# The last 6 variables are redundant
graz=graz[,1:27]
# Since we have some factors (here 0-1 variables like precipitation yes/no)
# we create the design matrix
# The design should not include other pollution levels since they will be highly
# correlated to NO2_DB, which is our response
attach(graz)
gr=model.matrix(NO2_DB~.,data=graz[,-c(3,4,5,6,7)])
yx=cbind(NO2_DB,gr)
# to use stepAIC we create a dataframe with all variables
yx=data.frame(yx)
modnull=lm(NO2_DB~1,data=yx)
stepAIC(modnull,
        direction="forward",
        scope=list(lower=~1, upper=yx[,2:23]))
```

Tuning the model

```
# The model with smallest AIC is this:
Step:  AIC=8808.33
NO2_DB ~ ltus_gk + lute_lagged + datum + wige + lufe_lagged +
    frost_laggedTRUE + ltus_gk_lagged + lufe + inversTRUE + wige_lagged +
    spg + spg_lagged + nied + nied01_laggedTRUE

                    Df Sum of Sq    RSS    AIC
<none>                           175518 8808.3
+ nied01TRUE         1   125.089 175393 8808.9
+ frostTRUE          1    69.742 175448 8809.6
+ nied_lagged        1    28.354 175490 8810.0
+ wind_laggedTRUE    1    19.019 175499 8810.1
+ windTRUE           1    12.572 175505 8810.2
+ invers_laggedTRUE  1     3.684 175514 8810.3
+ lute               1     0.182 175518 8810.3

Coefficients:
      (Intercept)            ltus_gk        lute_lagged              datum
        1.590e+02         -1.690e+00         -6.723e-01         -5.215e-08
             wige        lufe_lagged   frost_laggedTRUE     ltus_gk_lagged
       -1.142e+01         -2.057e-01          5.111e+00         -4.276e-01
             lufe         inversTRUE        wige_lagged                spg
       -1.077e-01          3.245e+00         -2.978e+00          1.412e-01
       spg_lagged               nied  nied01_laggedTRUE
        2.078e-01          8.943e-02         -7.771e-01
```

Tuning the model

```
# Here a summary of the chosen model
lm(formula = NO2_DB ~ ltus_gk + lute_lagged + datum + wige +
    lufe_lagged + frost_laggedTRUE + ltus_gk_lagged + lufe +
    inversTRUE + wige_lagged + spg + spg_lagged + nied + nied01_laggedTRUE,
    data = yx)

Coefficients:
                    Estimate Std. Error t value Pr(>|t|)
(Intercept)        1.590e+02  6.692e+00  23.764  < 2e-16 ***
ltus_gk           -1.690e+00  2.307e-01  -7.325 3.49e-13 ***
lute_lagged       -6.723e-01  3.659e-02 -18.375  < 2e-16 ***
datum             -5.215e-08  4.100e-09 -12.719  < 2e-16 ***
wige              -1.142e+01  1.080e+00 -10.583  < 2e-16 ***
lufe_lagged       -2.057e-01  2.971e-02  -6.923 5.98e-12 ***
frost_laggedTRUE   5.111e+00  9.025e-01   5.663 1.71e-08 ***
ltus_gk_lagged    -4.276e-01  1.729e-01  -2.473 0.013483 *
lufe              -1.077e-01  2.938e-02  -3.666 0.000253 ***
inversTRUE         3.245e+00  9.642e-01   3.365 0.000780 ***
wige_lagged       -2.978e+00  1.069e+00  -2.786 0.005391 **
spg                1.412e-01  1.021e-01   1.383 0.166903
spg_lagged         2.078e-01  1.019e-01   2.040 0.041534 *
nied               8.943e-02  5.512e-02   1.623 0.104843
nied01_laggedTRUE -7.771e-01  5.484e-01  -1.417 0.156601
---

Residual standard error: 9.522 on 1936 degrees of freedom
Multiple R-squared:  0.5601,Adjusted R-squared:  0.5569
F-statistic: 176.1 on 14 and 1936 DF,  p-value: < 2.2e-16
```

Tuning the model

```
# Let's try to run in both directions with stepAIC.
# This is slightly different from the "mixed" approach we described earlier.
# In each step we add or remove a variable, depending on what is leading to smallest AIC.
modnull=lm(NO2_DB~1,data=yx)
stepAIC(modnull,
        direction="both",
        scope=list(lower=~1, upper=yx[,2:23]))
```

Tuning the model

```
# Here a summary of the chosen model
# The variable spg has been kicked out
lm(formula = NO2_DB ~ ltus_gk + lute_lagged + datum + wige +
    lufe_lagged + frost_laggedTRUE + ltus_gk_lagged + lufe +
    inversTRUE + wige_lagged + spg_lagged + nied + nied01_laggedTRUE,
    data = yx)

Coefficients:
                    Estimate Std. Error t value Pr(>|t|)
(Intercept)        1.597e+02  6.678e+00  23.908  < 2e-16 ***
ltus_gk           -1.706e+00  2.304e-01  -7.402 1.99e-13 ***
lute_lagged       -6.681e-01  3.647e-02 -18.319  < 2e-16 ***
datum             -5.219e-08  4.101e-09 -12.725  < 2e-16 ***
wige              -1.056e+01  8.777e-01 -12.026  < 2e-16 ***
lufe_lagged       -2.064e-01  2.971e-02  -6.946 5.13e-12 ***
frost_laggedTRUE   5.075e+00  9.024e-01   5.624 2.13e-08 ***
ltus_gk_lagged    -4.500e-01  1.722e-01  -2.613 0.009036 **
lufe              -1.112e-01  2.927e-02  -3.800 0.000149 ***
inversTRUE         3.158e+00  9.624e-01   3.282 0.001051 **
wige_lagged       -3.117e+00  1.065e+00  -2.928 0.003455 **
spg_lagged         2.258e-01  1.011e-01   2.234 0.025585 *
nied               1.080e-01  5.346e-02   2.021 0.043438 *
nied01_laggedTRUE -7.830e-01  5.485e-01  -1.428 0.153574
---

Residual standard error: 9.524 on 1937 degrees of freedom
Multiple R-squared:  0.5596,Adjusted R-squared:  0.5567
F-statistic: 189.4 on 13 and 1937 DF,  p-value: < 2.2e-16
```

Tuning the model

```
# We try k-fold-cross-validation with k=10
k=10
set.seed(78)
# randomly assign each observration to one group
folds=sample(1:k,nrow(graz),replace=TRUE)
# For different models we would like to compute
# the test error rate. We get for each model k test errors.
# Altogether we consider 22 models, corresponding to the 22 parameters
# and selected by the order of the stepAIC.
nmodels=22
cv.errors=matrix(0,k,nmodels)
# Here is main loop which fills the cv.errors matrix
for(i in 1:nmodels){
  # choose the i-th model
  # we do stepAIC with k=0, which means that we add no penality!
  # This is because otherwise procedure stops when the lowest AIC is reached.
  modnull=lm(NO2_DB~1,data=yx)
  mod=formula(stepAIC(modnull,
          direction="forward",
          scope=list(lower=~1, upper=yx[,2:23]),
          steps=i,k=0)$terms)
  # For each left out set compute test error
  for(l in 1:k){
    submod=lm(mod,data=subset(yx,folds!=l))
    diff=predict(submod,yx)-yx$NO2_DB
    cv.errors[l,i]=mean(diff[folds==l]^2)
  }
}
```

Tuning the model

```
colMeans(cv.errors)
[1] 134.51339 113.71581 106.40695 101.11461  94.44320  93.42821  92.92294
 [8]  92.50313  92.12410  91.92302  91.80089  91.93038  91.94079  91.98110
[15]  92.09542  92.26614  92.29978  92.42211  92.46759  92.51910  92.70323
[22]  92.70323

which.min(colMeans(cv.errors))
# the model with 11 components is the one with smallest error
11
min(colMeans(cv.errors))
# the out-of-sample mean square test error is
91.80089
```

Shrinkage methods

The idea here is to penalize large values in the regression coefficients β in the linear regression

$$Y = X\beta + \varepsilon.$$

This means that we shrink β. This is a so-called regularization technique. It turns out that shrinking coefficients, reduces their variance.

We consider ridge regression and the lasso.

Shrinkage methods

For the ridge regression we replace the least squares criterion by

$$\|Y - X\beta\|^2 + \lambda\|\beta\|^2 = RSS + \lambda\|\beta\|^2.$$

Here λ is a tuning parameter. The bigger $\lambda \geq 0$, the more we shrink.

Hence our estimator $\hat{\beta}^{\text{Ridge}} = \text{argmin}_\beta \|Y - X\beta\|^2 + \lambda\|\beta\|^2$.

When $\lambda = 0$, we get the usual OLS estimator.

When $\lambda \to \infty$, the estimator converges to 0.

Shrinkage methods

The optimal λ may be chosen by cross-validation.

It can be shown (exercise) that

$$\hat{\beta}^{\text{Ridge}} = (X'X + \lambda I)^{-1} X'Y$$

and that the minimization problem is equivalent to

$$\min_{\beta} \|Y - X\beta\|^2 \quad \text{subject to} \quad \|\beta\|^2 \leq s,$$

for some $s = s(\lambda)$.

Shrinkage methods

Shrinking β increases the bias of our estimator, but decreases the variance.

By tuning λ, we can try to find an optimal bias-variance trade-off.

By constraining $\|\beta\|$ we have a kind of variable selection with the estimation. When p is large, most of the coefficients must be very small.

Shrinkage methods

```
# We try ridge regression on the graz data
install.packages("glmnet")
library(glmnet)
# we recall that
# gr=model.matrix(NO2_DB~.,data=graz[,-c(3,4,5,6,7)])
# alpha=0 means ridge, alpha=1 means lasso
# This is how to fit a rigde regression with the 50 values for lambda
ridge.mod=glmnet(x=gr,y=yx$NO2_DB,alpha=0)
# Here is how to get the fitted values for lambda=s
predict(ridge.mod,s=0,newx=gr,type="response")

# The goal is to determine an optimal lambda by k-fold cross-validation
k=10
set.seed(78)
# randomly assign each observration to one group
folds=sample(1:k,nrow(graz),replace=TRUE)
# for different values of lambda we would like to compute
# the test error rate. Each of those errors we get k-times
lambda=(0:49/10)^10
nlambda=length(lambda)
cv.errors=matrix(0,k,nlambda)
for(i in 1:nlambda){
  for(l in 1:k){
    ridge.mod=glmnet(x=gr[folds!=l,],y=yx$NO2_DB[folds!=l],alpha=0)
    diff=predict(ridge.mod,newx=gr[folds==l,],s=lambda[i])-yx$NO2_DB[folds==l]
    cv.errors[l,i]=mean(diff^2)
  }
}
```

Shrinkage methods

In this example no shrinkage is chosen!

```
min(colMeans(cv.errors))
92.8
which.min(colMeans(cv.errors))
1
colMeans(cv.errors)
 [1]  92.9  92.9  92.9  92.9  92.9  92.9  92.9  92.9  92.9  92.9  93.0  94.1  97.4
[14] 103.4 113.0 126.4 142.8 159.5 173.7 184.2 191.3 196.0 198.9 200.8 202.0 204.3
[27] 204.3 204.3 204.3 204.3 204.3 204.3 204.3 204.3 204.3 204.3 204.3 204.3 204.3
[40] 204.3 204.3 204.3 204.3 204.3 204.3 204.3 204.3 204.3 204.3 204.3
```

Shrinkage methods

The latter property is even more striking when we use the so-called lasso (least absolute shrinkage and selection operator) estimator:

$$\hat{\beta}^{\text{lasso}} = \text{argmin}_{\beta} \| Y - X\beta \|^2 + \lambda \|\beta\|_1,$$

where $\|\beta\|_1 = \sum_{i=1}^{p} |\beta_i|$.

The minimization problem is equivalent to

$$\min_{\beta} \| Y - X\beta \|^2 \quad \text{subject to} \quad \|\beta\|_1 \leq s,$$

for some $s = s(\lambda)$.

Shrinkage methods

By the shape of the unit ball with respect to the norm $\|\cdot\|_1$, we obtain that several coefficients of $\hat{\beta}^{\text{lasso}}$ might be exactly equal to zero, which then gives rise to a real variable selection.

Shrinkage methods

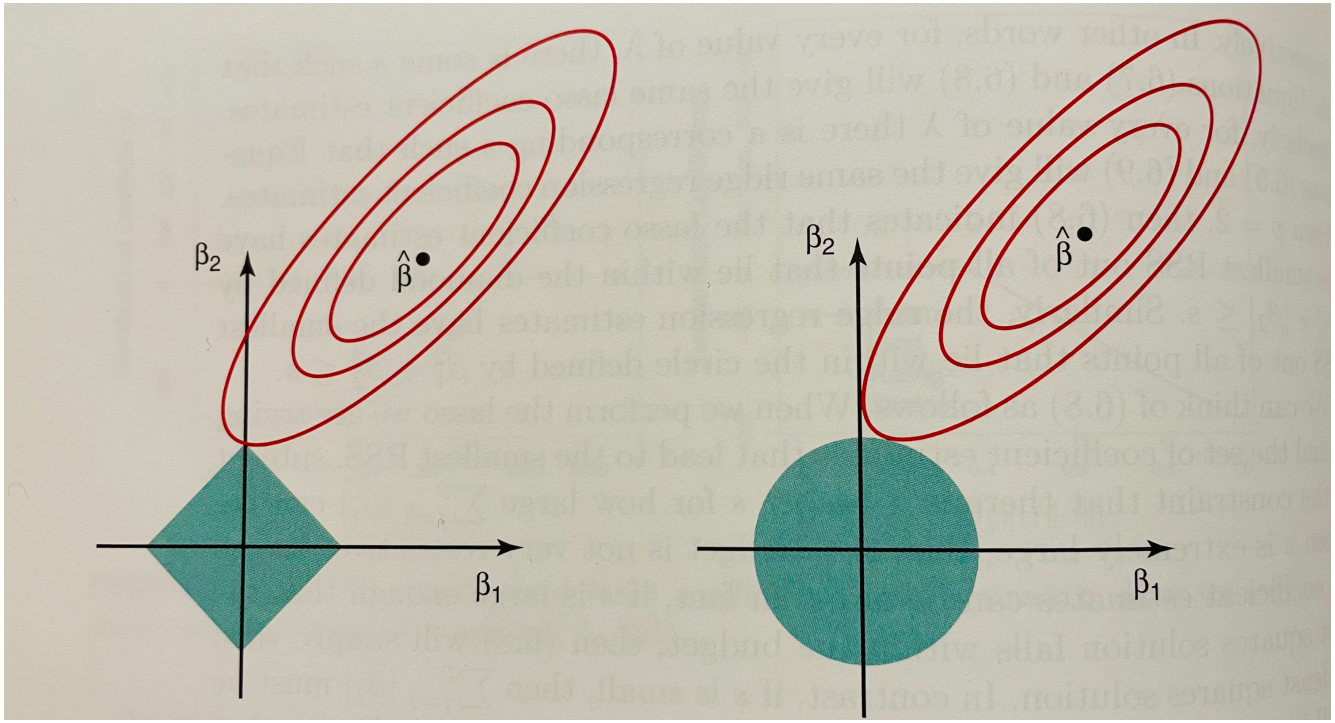


Figure 1: $\hat{\beta}$ is the OLS. Red contours are values of β with same SSR.

Shrinkage methods

```
# We try lasso on the graz data
# the setting is the same as with ridge
# The goal is to determine an optimal lambda by k-fold cross-validation
k=10
set.seed(78)
# randomly assign each observation to one group
folds=sample(1:k,nrow(graz),replace=TRUE)
# for different values of lambda we would like to compute
# the test error rate. Each of those errors we get k-times
lambda=(0:49/50)^10
nlambda=length(lambda)
cv.errors=matrix(0,k,nlambda)
for(i in 1:nlambda){
  for(l in 1:k){
# now take alpha=1
    lasso.mod=glmnet(x=gr[folds!=l,],y=yx$NO2_DB[folds!=l],alpha=1)
    diff=predict(lasso.mod,newx=gr[folds==l,],s=lambda[i])-yx$NO2_DB[folds==l]
    cv.errors[l,i]=mean(diff^2)
  }
}
```

Shrinkage methods

```
min(colMeans(cv.errors))
[1] 92.5
> which.min(colMeans(cv.errors))
[1] 38
> lambda[38]
[1] 0.0492
> colMeans(cv.errors)
 [1] 92.6 92.6 92.6 92.6 92.6 92.6 92.6 92.6 92.6 92.6 92.6 92.6 92.6 92.6 92.6
[16] 92.6 92.6 92.6 92.6 92.6 92.6 92.6 92.6 92.6 92.6 92.6 92.6 92.6 92.6 92.6
[31] 92.6 92.6 92.6 92.6 92.6 92.5 92.5 92.5 92.5 92.5 92.6 92.7 92.8 92.9 93.2
[46] 93.6 94.0 94.7 95.8 97.4
> predict(lasso.mod,newx=gr,s=lambda[8],type="coefficient")
(Intercept)        1.58e+02
datum             -5.13e-08
lute               .
lufe              -1.11e-01
wige              -1.09e+01
...
...

nied01_laggedTRUE -7.17e-01
ltus_gk_lagged    -5.58e-01
invers_laggedTRUE -4.68e-01
frost_laggedTRUE   3.42e+00
wind_laggedTRUE    2.23e-01
```

Shrinkage methods

An important feature of lasso or ridge is that they can deal with high-dimensional settings. With this we mean in particular situations in which $p > n$ or $p \gg n$.

```
> # We try the lasso on simulated data
> set.seed(78)
> n=200
> p=500
> X=matrix(rnorm(n*p),ncol=p,nrow=n)
> t=sample(1:p,20)
> beta=rep(1,p)
> # we set all but 20 randomly selected coefficients of beta equal to zero
> beta[-t]=0
> eps=rnorm(n)
> Y=X%*%beta+eps
```

Shrinkage methods

```
> # Again we search the optimal s (lambda) by k-fold cross-validation
> k=5
> # randomly assign each observration to one group
> folds=sample(1:k,nrow(X),replace=TRUE)
> # for different values of lambda we would like to compute
> # the test error rate.
> lambda=(0:49/50)^10
> nlambda=length(lambda)
> cv.errors=matrix(0,k,nlambda)
> for(i in 1:nlambda){
+   for(l in 1:k){
+     lasso.mod=glmnet(x=X[folds!=l,],y=Y[folds!=l],alpha=1)
+     diff=predict(lasso.mod,newx=X[folds==l,],s=lambda[i],type="response")-Y[folds==l]
+     cv.errors[l,i]=mean(diff^2)
+   }
+ }
```

Shrinkage methods

```
> min(colMeans(cv.errors))
[1] 1.88
> which.min(colMeans(cv.errors))
[1] 38
> colMeans(cv.errors)
 [1]  2.14  2.14  2.14  2.14  2.14  2.14  2.14  2.14  2.14  2.14  2.14  2.14  2.14
[14]  2.14  2.14  2.14  2.14  2.14  2.14  2.14  2.14  2.14  2.14  2.14  2.14  2.14
[27]  2.14  2.14  2.14  2.14  2.14  2.14  2.14  2.14  2.06  1.98  1.92  1.88  1.92
[40]  2.01  2.16  2.44  2.90  3.62  4.69  6.33  8.55 10.80 12.97 14.92
>
# Remember that the beta's where all =1.
# We check which estimated coefficients were larger than 1/2.
> which(as.vector(predict(ridge.mod,newx=X,s=lambda[38],type="coefficients"))[-1]>=1/2)
 [1]  17  18  36  60  85  90 123 147 151 185 198 204 269 287 290 299 332 341 395
[20] 482
# And now we check which beta coeffs were =1.
> which(beta==1)
 [1]  17  18  36  60  85  90 123 147 151 185 198 204 269 287 290 299 332 341 395
[20] 482
```

Extensions

Large p settings can also occur, when we consider interaction effects or consider polynomials of our covariates.

Hence covariates of the type

$$x_i^k \quad \text{or} \quad x_i x_j \quad \text{or} \quad x_i x_j x_k \quad \text{etc.}$$

are included.

Extensions

The Auto data example (see Ch 7).

```
> mod=lm(mpg~weight*year+poly(horsepower,3),data=Auto)
> summary(mod)

Call:
lm(formula = mpg ~ weight * year + poly(horsepower, 3), data = Auto)

Residuals:
    Min      1Q  Median      3Q     Max
-8.1020 -1.8693 -0.0486  1.4300 12.0411

Coefficients:
                       Estimate Std. Error t value Pr(>|t|)
(Intercept)          -8.138e+01  1.344e+01  -6.056 3.32e-09 ***
weight                1.763e-02  4.774e-03   3.692 0.000255 ***
year                  1.597e+00  1.771e-01   9.022  < 2e-16 ***
poly(horsepower, 3)1 -2.394e+01  6.741e+00  -3.551 0.000431 ***
poly(horsepower, 3)2  2.304e+01  3.461e+00   6.656 9.70e-11 ***
poly(horsepower, 3)3 -1.218e+01  3.046e+00  -4.001 7.58e-05 ***
weight:year          -3.064e-04  6.305e-05  -4.860 1.71e-06 ***
---

Residual standard error: 2.944 on 385 degrees of freedom
Multiple R-squared:  0.8599,        Adjusted R-squared:  0.8577
F-statistic: 393.9 on 6 and 385 DF,  p-value: < 2.2e-16
```

Principal components regression

The `graz` pollution data contain 48 daily measurements for `PM10` and for temperatures in Graz and Kalkleiten.

Suppose we would like to predict the daily mean of `PM10` on day t based on the 48 `PM10` values of day $t-1$ and corresponding 48 temperatures from Graz and 48 temperatures from Kalkleiten.

That would give a regression with $48 * 3 = 144$ covariates.

We use instead principal components regression.

Principal components regression

To this end we transform each of the lagged "curves" by PCA to 3 scores.

$\Longrightarrow$ We get a design X with 9 columns.

Now we regress the daily means onto X.

Principal components regression

```
# We want to predict daily mean of PM10 by the 48 lagged PM10 30 min.
# 48 lagged T.GR and T.KL 30 min.
# We read the PM10 data.
pm=read.table("data.txt",header=TRUE,sep=",")

# We attribute to the variable "Date" a date format.
pm$Date=as.Date(pm$Date, "%d.%m.%Y")

# We attribute to the varibles PM.GR, T.GR and T.KL a time series format.
pm.timeseries <- xts(pm[,c(4,6,7)], as.Date(pm$Date))

# Compute daily averages for PM.GR and this is our response Y.
Y=apply.daily(pm.timeseries$PM.GR, mean)

# Put PM.GR, T.GR and T.KL in a data matrix.
n=length(pm.daily.mean)
matrix.PM.GR=matrix(0,n,48)
for(i in 1:length(pm.daily.mean))matrix.PM.GR[i,]=pm.timeseries$PM.GR[((i-1)*48+1):(i*48)]
matrix.T.GR=matrix(0,n,48)
for(i in 1:length(pm.daily.mean))matrix.T.GR[i,]=pm.timeseries$T.GR[((i-1)*48+1):(i*48)]
matrix.T.KL=matrix(0,n,48)
for(i in 1:length(pm.daily.mean))matrix.T.KL[i,]=pm.timeseries$T.KL[((i-1)*48+1):(i*48)]
```

Principal components regression

```
# We compute 3 PC's for each of these vectors.
pc.PM.GR=princomp(matrix.PM.GR)$scores[,1:3]
colnames(pc.PM.GR)=c("PM.GR PC1","PM.GR PC2","PM.GR PC3")
pc.T.GR=princomp(matrix.T.GR)$scores[,1:3]
colnames(pc.T.GR)=c("T.GR PC1","T.GR PC2","T.GR PC3")
pc.T.KL=princomp(matrix.T.KL)$scores[,1:3]
colnames(pc.T.KL)=c("T.KL PC1","T.KL PC2","T.KL PC3")

# We compute our design matrix from the scores.
X=cbind(pc.PM.GR,pc.T.GR,pc.T.KL)

# And now the lag-1 version to make prediction meaningful.
X=rbind(rep(0,9),X[-n,])
yx=data.frame(Y,X)
mod0=lm(PM.GR~1,data=yx)
modfull=lm(PM.GR~.,data=yx)
stepAIC(mod,direction = "both",scope=list(lower=mod0,upper=modfull))
```

Principal components regression

```
# This was the selected model by AIC
modfin=lm(PM.GR ~ PM.GR.PC1 + T.KL.PC1 + T.GR.PC3 + PM.GR.PC2 + T.GR.PC1 + T.KL.PC2, data = yx)
summary(modfin)

Residuals:
    Min      1Q  Median      3Q     Max
-42.662  -6.983  -0.339   5.512  82.047

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) 41.904404   0.981710  42.685  < 2e-16 ***
PM.GR.PC1    0.106417   0.007589  14.022  < 2e-16 ***
T.KL.PC1    -0.235530   0.069086  -3.409 0.000809 ***
T.GR.PC3     1.001701   0.290163   3.452 0.000697 ***
PM.GR.PC2   -0.116534   0.017732  -6.572 5.49e-10 ***
T.GR.PC1     0.155609   0.080659   1.929 0.055321 .
T.KL.PC2    -0.228623   0.137167  -1.667 0.097353 .
---

Residual standard error: 13.24 on 175 degrees of freedom
Multiple R-squared:  0.6627,        Adjusted R-squared:  0.6511
```

We remark the increase in the Adjusted R^2 compared to regression on daily averages from 0.56 to 0.65.

Issues

We have already discussed in Chapter 7 a few diagnostic tools to check model assumptions:

1. Check normality assumptions with qq-plots of residuals.
2. Plot residuals against covariates to see non-linear effects.

We conclude this section by a few further potential issues which should be checked.

Issues

Outliers: The observation y_i is far from the predicted value.

This can have an impact on the model parameters and/or influence the modelfit, e.g. by a much smaller R^2.

In such cases it may be sensible to eliminate this observation. This should, however, be done with care. Could also indicate a problem with overall model.

Issues

High leverage points: The observation x_i is far from the usual values.

This can have an impact on the model parameters and/or influence the modelfit, e.g. by a much smaller R^2.

In such cases it may be sensible to eliminate this observation. This should, however, be done with care. Could also indicate a problem with overall model.

Issues

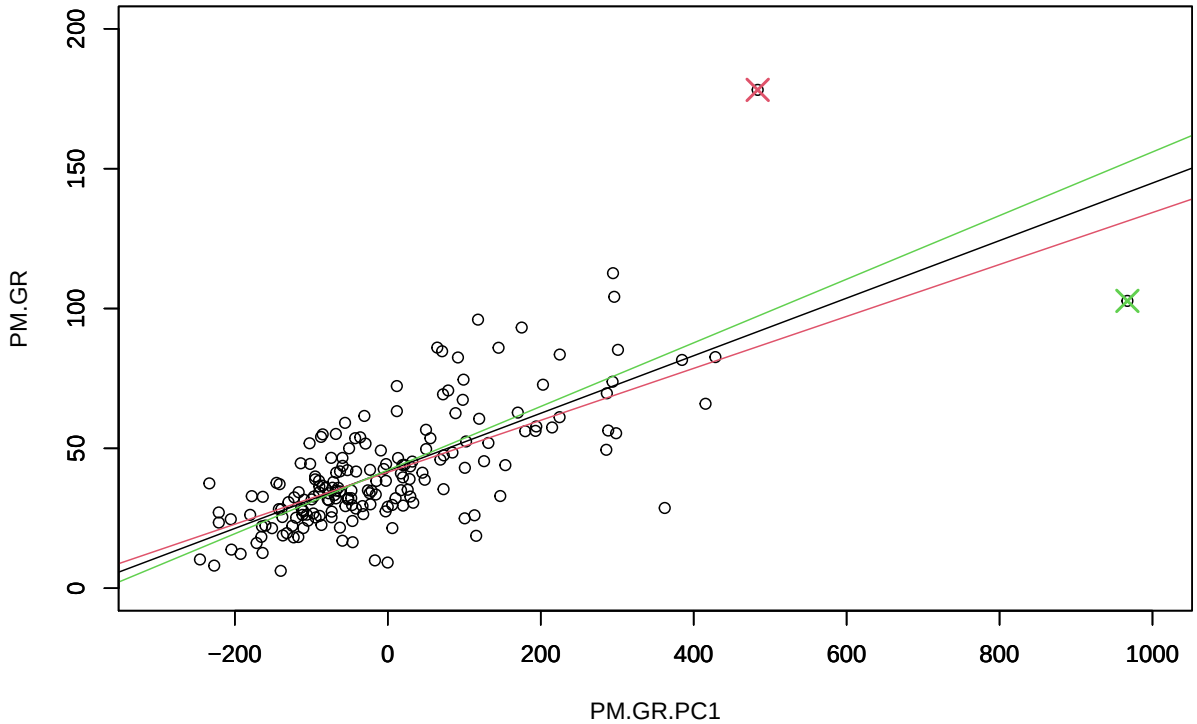


Figure 2: Regression y =PM.GR on x =PM.GR.PC1. Black: regression line with all data. Red: regression line removing outlier. Green: regression line removing high leverage point.

Issues

Correlated residuals: Especially with time series data it may happen, that residuals are correlated.

This is particularly problematic when we run our F-tests, because the distribution theory is based on the assumption of iid normal errors. While violation of normality becomes less problematic if sample size increases (because of CLT), correlated residuals will remain problematic. It might be helpful to include lagged values as covariates.

Hence we should check the auto-covariance function of the residuals.

Issues

Colinearity: The columns of our design matrix X need to be linearly independent, in order that we can compute

$$\hat{\beta} = (X'X)^{-1}X'Y.$$

If not, we run into a singularity issue.

Actually, when we go back to our projection approach we need β such that

$$X'Y = X'X\hat{\beta}.$$

If $X'X$ is not full rank we have multiple solutions, all leading to the same $\hat{\mu} = X\hat{\beta}$.

Hence the issue is not estimating μ but estimating β. When $X'X$ is close to singular (i.e. columns are 'almost' linearly dependent), we have the problem of very unstable estimates for β. Hence a small change in X can lead to very different β.

Issues

Colinearity can be checked with the variance inflation factor.

$$\text{VIF}(\hat{\beta}_j) = \frac{1}{1 - R^2(x_j|x_{-j})},$$

where $R^2(x_j|x_{-j})$ is the R^2 we get by regressing x_j onto the remaining columns of X.